

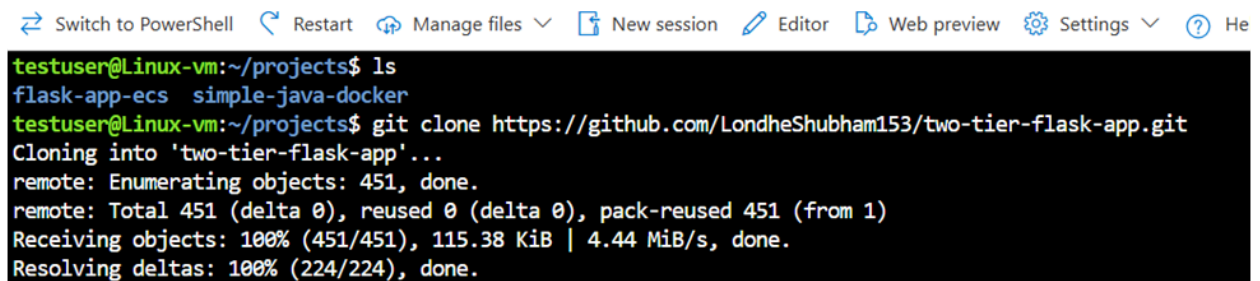
Dockerizing a Two-Tier Flask Application with MySQL Integration

Recently, I worked on containerizing a Flask web application with a MySQL backend. The goal was to run both containers on the same Docker network to ensure communication between the application and the database.

Step-by-Step Breakdown:

- ♦ **Step 1: Clone the Project & Create a Flask Image** I started by cloning a prebuilt repo:

```
git clone https://github.com/LondheShubham153/two-tier-flask-app
cd two-tier-flask-app
```



```
testuser@Linux-vm:~/projects$ ls
flask-app-ecs  simple-java-docker
testuser@Linux-vm:~/projects$ git clone https://github.com/LondheShubham153/two-tier-flask-app.git
Cloning into 'two-tier-flask-app'...
remote: Enumerating objects: 451, done.
remote: Total 451 (delta 0), reused 0 (delta 0), pack-reused 451 (from 1)
Receiving objects: 100% (451/451), 115.38 KiB | 4.44 MiB/s, done.
Resolving deltas: 100% (224/224), done.
```

Then created a Dockerfile to build the Flask app image:

 Switch to PowerShell
  Restart
  Manage files
  New session
  Editor
  Web preview
 

```

## Use an official Python runtime as the base image
FROM python:3.9-slim

# Set the working directory in the container
WORKDIR /app

# install required packages for system
RUN apt-get update \
    && apt-get upgrade -y \
    && apt-get install -y gcc default-libmysqlclient-dev pkg-config \
    && rm -rf /var/lib/apt/lists/*

# Copy the requirements file into the container
COPY requirements.txt .

# Install app dependencies
RUN pip install mysqlclient
RUN pip install --no-cache-dir -r requirements.txt

# Copy the rest of the application code
COPY . .

# Specify the command to run your application
CMD ["python", "app.py"]

~
~
~
~

```

Then I built the image:

```
docker build -t two-tier-backend .
```

```

testuser@Linux-vm: ~/projects/two-tier-flask-app$ docker build -t two-tier-backend .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

```

```
testuser@Linux-vm:~/projects/two-tier-flask-app$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
two-tier-backend	latest	a4c948eaaa55	2 hours ago	399MB
<none>	<none>	dbf824330909	2 hours ago	399MB
<none>	<none>	85872886ab4f	2 hours ago	391MB
flask-app	latest	eb9e685deefd	15 hours ago	137MB
java-app	latest	54ae92344e2d	15 hours ago	326MB
<none>	<none>	4cd3ee61aad8	15 hours ago	326MB
mysql	latest	2c849dee4ca9	2 weeks ago	859MB
python	3.9-slim	9a041530811d	3 weeks ago	126MB
hello-world	latest	74cc54e27dc4	3 months ago	10.1kB
openjdk	17-jdk-alpine	264c9bdce361	3 years ago	326MB

✓ Step 2: Create a Custom Docker Network

To allow both containers to talk to each other using internal container names:

```
docker network create two-tier
```

```
testuser@Linux-vm:~/projects/two-tier-flask-app$ docker network create two-tier -d bridge
21b3a1ddfb8a08305d38451cfb995c451490b697d8e26f8add554ec3fdee37ee
testuser@Linux-vm:~/projects/two-tier-flask-app$
```

✓ Step 3: Run MySQL Container

♦ **Step 3: Run the MySQL Container in the Network** Used the prebuilt MySQL image and passed required environment variables. This connects the container to the `two-tier` network:

```
docker run -d --name mysql --network two-tier \
-e MYSQL_ROOT_PASSWORD=root \
-e MYSQL_DATABASE=devops \
mysql
```

```
testuser@Linux-vm:~/projects/two-tier-flask-app$ docker run -d --name mysql --network two-tier -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=devops mysql
```

✓ Step 4: Run Flask App Container

Now, I deployed the Flask app in the same network and connected it to the MySQL container using the container name as host (`MYSQL_HOST=mysql`):

```
docker run -d -p 5000:5000 --network two-tier \
-e MYSQL_HOST=mysql \
-e MYSQL_USER=root \
-e MYSQL_PASSWORD=root \
-e MYSQL_DB=devops \
two-tier-backend:latest
```

📌 Note: `MYSQL_HOST=mysql` works because Docker resolves container names as DNS entries on the same network.

```
testuser@Linux-vm:~/projects/two-tier-flask-app$ docker run -d -p 5000:5000 --network two-tier -e MYSQL_HOST=mysql -e MYSQL_USER=root -e MYSQL_PASSWORD=root -e MYSQL_DB=devops two-tier-backend:latest
```

✓ Step 5: Verify the Containers

Checked if both containers were running successfully:

`docker ps`

```
testuser@Linux-vm:~/projects/two-tier-flask-app$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
cfa2db7b37fd   two-tier-backend:latest   "python app.py"         39 minutes ago   Up 39 minutes   0.0.0.0:5000->5000/tcp, :::5000->5000/tcp   quirky_driscoll
5be71d123b93   mysql          "docker-entrypoint.s..." 42 minutes ago   Up 42 minutes   3306/tcp, 33060/tcp                 mysql
```

Arifa

Step 6: Debug via Logs Troubleshoot using container logs (helpful in case of DB connection issues):

`docker logs <container-id>`

```
Switch to PowerShell Restart Manage files New session Editor Web preview Settings Help
testuser@Linux-vm:~/projects/two-tier-flask-app$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED        STATUS        PORTS                               NAMES
cfa2db7b37fd   two-tier-backend:latest "python app.py"         28 minutes ago Up 28 minutes 0.0.0.0:5000->5000/tcp, :::5000->5000/tcp   quirky_driscoll
5be71d123b93   mysql                 "docker-entrypoint.s..." 31 minutes ago Up 31 minutes 3306/tcp, 33060/tcp                    mysql
testuser@Linux-vm:~/projects/two-tier-flask-app$ docker logs cfa2db7b37fd
* Serving Flask app 'app' (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.19.0.3:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 118-689-089
103.245.194.80 - - [01/May/2025 10:31:13] "GET / HTTP/1.1" 200 -
103.245.194.80 - - [01/May/2025 10:31:35] "GET /favicon.ico HTTP/1.1" 404 -
103.245.194.80 - - [01/May/2025 10:32:43] "POST /submit HTTP/1.1" 200 -
testuser@Linux-vm:~/projects/two-tier-flask-app$
```

✓ Step 6: Inspect Docker Network

Check if both containers are connected to the `two-tier` network:

`docker network inspect two-tier`

Arifa

You should see both `mysql` listed under the `Containers` section.

```
testuser@Linux-vm:~/projects/two-tier-flask-app$ docker network inspect two-tier
```

```
[
  {
    "Name": "two-tier",
    "Id": "21b3a1ddfb8a08305d38451cfb995c451490b697d8e26f8add554ec3fdee37ee",
    "Created": "2025-05-01T08:11:11.411709524Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.19.0.0/16",
          "Gateway": "172.19.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    }
  }
]
```

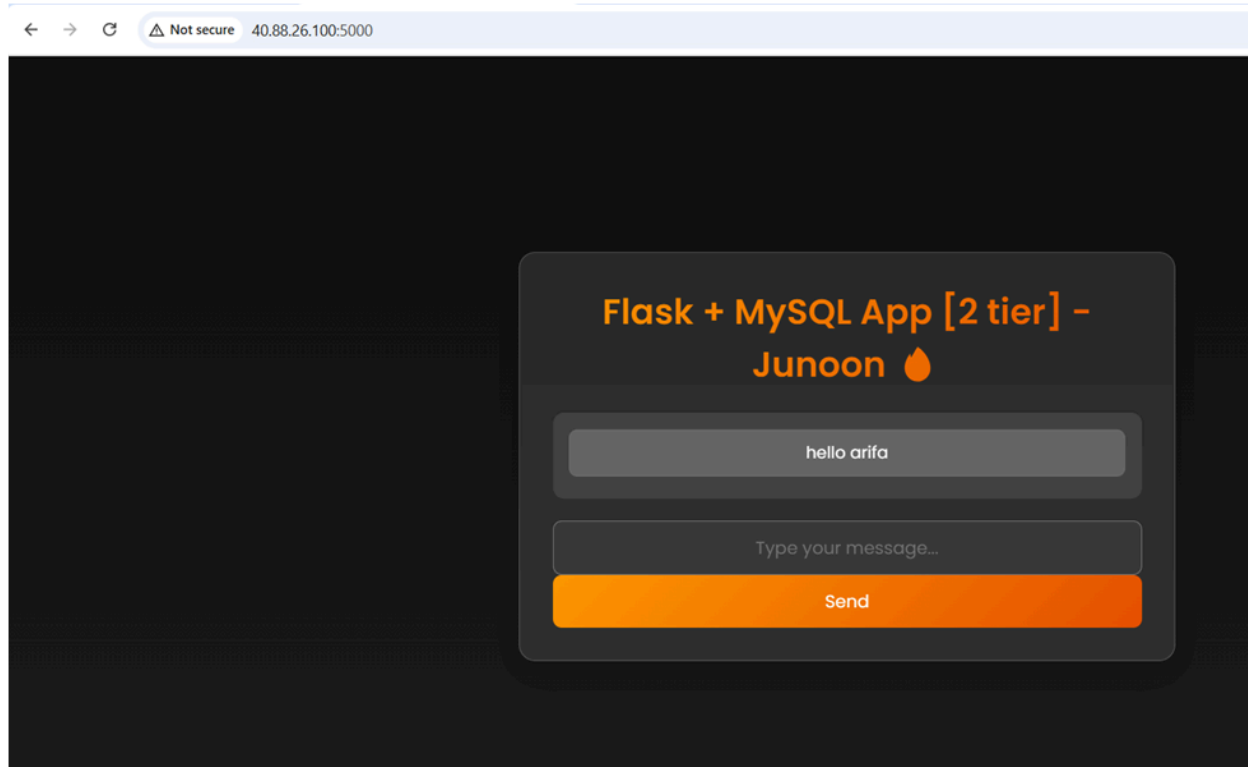
Switch to PowerShell Restart Manage files New session Editor Web preview Settings Help

```
    "Subnet": "172.19.0.0/16",
    "Gateway": "172.19.0.1"
  }
],
"Internal": false,
"Attachable": false,
"Ingress": false,
"ConfigFrom": {
  "Network": ""
},
"ConfigOnly": false,
"Containers": {
  "5be71d123b93b1346bcc1c67578e9ecef75e97e989915f295bab97bd24fa2d": {
    "Name": "mysql",
    "EndpointID": "5dee491a238ad8ca2ff47721e294291b0c2db8cf881360f7e50f7790c1dd2505",
    "MacAddress": "02:42:ac:13:00:02",
    "IPv4Address": "172.19.0.2/16",
    "IPv6Address": ""
  },
  "cfa2db7b37fd155ffa4a901849202273d51b6ede96eabdb1190a229d58a5700d": {
    "Name": "quirky_driscoll",
    "EndpointID": "1fc38eca833107cb7adc51e3bdceacb8925ef840464a3275815351525379de6c",
    "MacAddress": "02:42:ac:13:00:03",
    "IPv4Address": "172.19.0.3/16",
    "IPv6Address": ""
  }
},
"Options": {},
"Labels": {}
}
```

✓ Step 7: Access the Flask App

Finally, open the app in your browser using your VM's public IP and port 5000:

`http://<your-vm-public-ip>:5000`



Make sure port 5000 is allowed through your vm's security group settings.

📖 Key Learnings:

- Use Docker networks for container-to-container communication
- Pass environment variables to containers for dynamic configuration
- Understand how internal DNS works inside Docker networks
- Build and run multi-container applications easily with Docker

💡 This setup forms the base for deploying containerized microservices and can be scaled further using orchestration tools like Docker Compose or Kubernetes.

#Docker #Flask #MySQL #DevOps #CloudNative #Python #Containerization #Microservices
#FullStackDev #TwoTierArchitecture #ProjectShowcase